CartelOS AI Architecture — Before vs After LangChain Migration

Audience: Non-technical stakeholders, reviewers, onboarding readers **Scope:** Live-call AI workflow before and after the LangChain/LangGraph migration **Audit basis:** Source tree + git history (`228caa6` → `a9a9d10` → `6dc25e7`) **Last audited:** 2026-06-15

1) Why this document exists

This document explains, in plain language, how CartelOS handled AI live-call workflows **before** LangChain and how the workflow works **after** the migration.

The goal is to make the change easy to understand for a non-technical reader:

- what the workflow looked like earlier,
- what changed after LangChain/LangGraph,
- which major functions and files were introduced or modified,
- what business and engineering benefits were achieved,
- and what technical debt still remains.

2) Simple executive summary

CartelOS originally handled live AI calls using a **custom, manual approach**:

- direct calls to OpenAI through raw HTTP requests,
- one large orchestration function for the whole turn,
- manual JSON parsing,
- and limited cancellation, validation, and observability.

After the migration, CartelOS moved much of the live-call path to a more structured architecture:

- **LangChain** for AI model access,
- **LangGraph** for step-by-step workflow orchestration,
- **Zod schemas** for reliable structured outputs,
- **conversation memory summaries** for longer calls,
- and **better cancellation handling** when users interrupt the assistant.

However, the migration is **not fully complete**. The WebSocket live streaming path now uses the new graph-based system, but some REST endpoints still use the older legacy path.

3) The big picture in one sentence

Before

A single large function handled almost everything manually.

After

The workflow is broken into clear steps, validated more safely, and is easier to observe, maintain, and extend.

4) Before LangChain — how the workflow used to work

In the old system, the AI call workflow was mostly built with hand-written logic.

Main characteristics of the old approach

1. **Direct OpenAI API calls** were made using raw `fetch(...)` requests.
2. **Streaming responses** were handled through manual SSE parsing.
3. **JSON responses** were parsed with custom helper functions.
4. A **single large orchestration function** managed validation, routing, AI generation, WebSocket events, persistence, and TTS behavior.
5. There was **no clean shared AI client layer**.
6. Multi-persona routing relied more on **heuristics and keyword matching**.
7. If the user interrupted the assistant, cancellation behavior was more fragile.

Old workflow in plain words

When the learner spoke, the backend would:

- receive transcript or audio,
- build the context again,
- decide who should respond,
- call OpenAI manually,
- parse the JSON manually,
- save the result,
- and stream or return the output.

This worked, but it made the system harder to scale and harder to maintain.

5) After LangChain — how the workflow works now

The new architecture keeps the same product behavior for users, but changes the internal implementation.

Main characteristics of the new approach

1. **LangChain ChatOpenAI** is now used instead of raw OpenAI fetch calls.
2. **LangGraph** runs the main WebSocket live-call workflow as a sequence of nodes.
3. **Zod schemas** validate structured outputs.
4. **Legacy parsers are still kept** as fallback safety nets.
5. **Conversation memory summaries** help long calls stay efficient.
6. **AbortController + turn generation logic** prevent older responses from leaking into newer turns.
7. **Context caching and retrieval** improve long-session efficiency.

New workflow in plain words

When the learner speaks now, the backend usually:

- validates the session,
- prepares the transcript and context,
- loads recent history,
- classifies the learner intent,
- picks the correct persona,
- isolates that persona’s context,
- starts the WebSocket stream,
- generates the AI response,
- saves the result,
- updates memory in the background,
- and triggers TTS in a non-blocking way.

This makes the system more modular and easier to reason about.

6) Before vs After workflow comparison

Area	Before	After
OpenAI access	Raw `fetch()` calls	LangChain `ChatOpenAI` wrappers
Workflow orchestration	One large monolithic function	LangGraph step-by-step workflow
Response validation	Manual JSON parsing	Zod schemas + structured output
Streaming	Manual SSE handling	LangChain streaming + existing reply extraction
Cancellation	Limited, fragile	Abort-aware turn supersession
Memory	Full history only	Rolling summaries for long calls
Retrieval	More static context loading	Cached context + retrieval after early turns
Maintainability	Hard to change	Easier to test and extend
Observability	Limited	Per-node graph logging
REST and WebSocket parity	Mostly same old path	WebSocket updated, REST still partly legacy

7) What exactly changed in the workflow

7.1 Earlier workflow

The earlier implementation was centered around a big function that did many jobs together:

- validate session,
- finalize transcript,
- build context,
- choose persona,
- call OpenAI,
- stream output,
- save messages,
- and schedule TTS.

This meant one function had too many responsibilities.

7.2 New workflow

The new implementation separates these responsibilities into smaller graph nodes.

Current LangGraph steps

```
validateSessionNode
prepareTurnNode
loadHistoryNode
classifyLearnerIntentNode
routePersonaNode
isolatePersonaContextNode
streamWebsocketStartNode
generateResponseNode
persistMessagesNode
generateTtsNode
```

Why this matters

Instead of debugging one giant block of logic, engineers can now inspect a specific step.

For example:

- if persona selection is wrong, inspect routing nodes,
- if streaming is slow, inspect generation and stream nodes,
- if persistence fails, inspect save-related nodes,
- if long calls lose context, inspect memory and retrieval.

8) Key functions — before and after

This section highlights the most important function changes in simple language.

8.1 `processLearnerTurnStreaming`

Before

This function itself contained most of the live streaming workflow.

After

It became a lightweight entry point that now delegates to:

- `runLiveCallTurnGraph(...)`

Meaning: the big logic was moved into a structured graph.

8.2 `runLiveCallTurnGraph`

Before

This function did not exist.

After

It became the central orchestration point for WebSocket turns.

Meaning: the live-call flow is now explicitly modeled as connected steps.

8.3 `generateLiveCallTurn`

Before

It depended on raw OpenAI calls and manual parsing.

After

It uses structured output with:

- `invokeStructuredLiveCallChat(...)`
- `LiveCallTurnSchema`
- fallback to `parseLiveCallAiJson(...)` if needed

Meaning: response handling became more reliable, but still keeps a safety fallback.

8.4 `streamLiveCallTurn`

Before

It used manual fetch + SSE parsing.

After

It streams via the shared LangChain client path.

Meaning: the transport layer is cleaner, but the final stream buffer is still parsed using the legacy JSON parser.

8.5 `parseLiveCallAiJson`

Before

This was the primary way to parse model output.

After

It is still present, but now mainly acts as:

- a fallback parser,
- and the final parser for completed streaming buffers.

Meaning: it was not removed because it still protects the system in edge cases.

8.6 `processLearnerTextReply`

Before

This was the main turn-processing function.

After

It still exists for some REST endpoints.

Meaning: this is one of the clearest signs that the migration is only partial.

8.7 `classifyLearnerMultiPersonaIntent`

Before

This function did not exist in the same modern form.

After

It adds semantic intent classification for multi-persona calls.

Meaning: persona routing is now smarter than simple keyword matching.

8.8 `updateConversationMemorySummary`

Before

This capability did not exist.

After

It creates a rolling memory summary every few learner turns.

Meaning: long conversations are easier to manage without stuffing the whole history into every prompt.

9) Files added or changed

Below is a simplified view of the most important file-level changes.

New shared AI foundation

- `backend/src/lib/ai/openai.client.js`

Shared LangChain OpenAI client layer.

- `backend/src/lib/ai/structured-output.js`

Structured output support with fallback parsing.

- `backend/src/lib/ai/text-chain.js`

Reusable LangChain text chain helpers.

- `backend/src/lib/ai/intake-graph.js`

Shared LangGraph intake pipeline factory.

New live-call LangChain/LangGraph layer

- `backend/src/services/live-call/graph/live-call-turn.graph.js`

Main LangGraph workflow for WebSocket turns.

- `backend/src/services/live-call/langchain/live-call-openai.client.js`

Live-call specific wrappers around shared AI client code.

- `backend/src/services/live-call/langchain/live-call-schemas.js`

Zod schemas for live-call outputs.

- `backend/src/services/live-call/langchain/live-call-structured-output.js`

Live-call structured output wrapper.

- `backend/src/services/live-call/langchain/live-call-chains.js`

Opening, analysis, and chain-based generation helpers.

- `backend/src/services/live-call/langchain/live-call-prompt-templates.js`

LangChain prompt templates around existing prompt wording.

- `backend/src/services/live-call/langchain/live-call-memory.service.js`

Rolling memory summary logic.

- `backend/src/services/live-call/langchain/live-call-learner-intent.service.js`

Intent classification for multi-persona scenarios.

- `backend/src/services/live-call/langchain/live-call-turn-abort.js`

Turn cancellation logic.

- `backend/src/services/live-call/langchain/live-call-graph-logger.js`

Graph-level logging and observability.

Important modified files

- `backend/src/services/live-call/live-call-ai.service.js`
- `backend/src/services/live-call/live-call.service.js`
- `backend/src/services/live-call/live-call-context.service.js`
- `backend/src/prompts/live-call-ai.prompt.js`
- `backend/src/ws/live-call.ws.js`

10) What business and engineering problems were solved

10.1 Cleaner architecture

The biggest improvement is architectural clarity. The codebase moved away from an oversized all-in-one turn handler.

10.2 Better reliability

Structured outputs with schemas reduce the chance of malformed AI responses causing silent failures.

10.3 Safer streaming behavior

User interruptions are now handled better through abort-aware turn management.

10.4 Better long-call handling

Memory summaries and retrieval reduce prompt bloat in long sessions.

10.5 Better maintainability

It is easier for new engineers to understand a pipeline of nodes than a huge monolithic function.

10.6 Better debugging

Graph node logs make it easier to locate latency or failure points.

11) What did not change completely

Even after the migration, some parts of the old system remain intentionally in place.

Still legacy or partially migrated

- REST turn endpoints still use `processLearnerTextReply`
- `parseLiveCallAiJson(...)` still exists
- stream completion still relies on legacy-style final parsing
- live-call pgvector embeddings still use raw fetch instead of LangChain
- some fallback opening/goodbye/template logic still remains

Why this matters

The migration was designed for **stability first**, not for deleting every old component at once.

That means the new architecture is stronger, but there is still a dual-path maintenance cost.

12) The most important “before vs after” answer for management

If someone asks, **“What really changed?”**, the simplest answer is:

> *CartelOS moved from a custom, hand-managed AI call pipeline to a more structured architecture where AI access, workflow orchestration, validation, cancellation, memory, and observability are handled in a cleaner and more scalable way.*

And if they ask, **“Did everything migrate?”**, the honest answer is:

> *No. The WebSocket live-call path is modernized, but some REST paths and fallback parsers still rely on legacy logic.*

13) Simple before vs after narrative

Before migration

The system behaved like a single operator trying to do everything alone: listen, think, answer, save data, and manage timing all inside one crowded block.

After migration

The system behaves more like a team with assigned roles: one step validates, another prepares context, another routes persona, another generates the answer, another saves it, and another handles TTS.

This is the clearest non-technical way to understand the architectural improvement.

14) Technical debt that still remains

The major remaining debt is not that the new system is weak — it is that **old and new systems currently coexist**.

Main remaining issues

1. **Dual turn paths** exist:
   - WebSocket uses LangGraph
   - REST still uses legacy imperative processing
1. **Legacy parsers still remain**:
   - useful for safety,
   - but still extra code to maintain.
1. **Streaming still uses custom extraction logic**:
   - structured output is not yet fully native during token flow.
1. **Embeddings are not fully standardized**:
   - live-call retrieval still uses raw fetch for embeddings.

Recommended future direction

- unify REST and WebSocket paths,
- reduce legacy fallback dependency over time,
- adopt stronger streaming structured output if stable,
- standardize embeddings implementation,
- and continue extending schema validation to other AI services.

15) Final conclusion

The LangChain/LangGraph migration is a meaningful architectural upgrade for CartelOS.

It improved:

- structure,
- reliability,
- observability,
- maintainability,
- and long-session handling.

At the same time, the migration was intentionally cautious. The team kept some old parsers and legacy flows to avoid breaking live-call behavior.

So the most accurate conclusion is:

CartelOS has successfully modernized its primary live-call workflow, but it is still in a hybrid state rather than a 100% fully migrated state.

16) Appendix — quick list of the most visible function changes

Newly added or newly central functions

- `runLiveCallTurnGraph`
- `invokeStructuredLiveCallChat`
- `updateConversationMemorySummary`
- `classifyLearnerMultiPersonaIntent`
- `loadContextForTurn`
- `beginServerTurn`

Functions that stayed but changed behavior

- `processLearnerTurnStreaming`
- `generateLiveCallTurn`
- `streamLiveCallTurn`
- `createLiveCallSession`

Functions still acting as legacy fallback

- `parseLiveCallAiJson`
- `processLearnerTextReply`
- fallback template/opening/goodbye helpers

17) One-page summary

Before

- manual API calls
- manual JSON parsing
- one large orchestration function
- weaker cancellation
- less visibility into step-level behavior

After

- LangChain for AI calls
- LangGraph for workflow orchestration
- Zod for structured validation
- memory summaries for long calls
- improved cancellation and turn supersession
- better observability and modularity

Still pending

- REST path unification
- full removal of legacy parsers
- streaming structured output improvements
- embeddings consistency

Prepared as a reader-friendly migration document based on the provided audit content.

Generated from the provided migration audit and rewritten into a reader-friendly PDF format.